센서 로그 파싱 & 시각화 자동화

JSON · Regex · NumPy · Matplotlib · Threading

cchyun@gmail.com

Part 1. 실습 개요 및 목표

실습 목표 — 로봇 로그 분석 자동화 시스템 구축

- **로봇이 운행 중 생성한 텍스트 로그 파일에서 센서 데이터를 추출·분석·시각화·자동화**

  - **배우는 것**
    - `config.json`으로 임계치(Threshold) 설정을 관리하는 방법
    - 정규표현식(Regex)으로 비정형 텍스트에서 데이터를 추출하는 방법
    - **NumPy**로 이상값(Anomaly)을 탐지하고 **Matplotlib**으로 시각화하는 방법
    - **멀티스레딩**으로 분석 작업을 백그라운드에서 자동화하는 방법

  - **만들어지는 결과물**
    - `sensor_report.png` — 이상값이 표시된 센서 데이터 그래프
    - 메인 루프와 분석 스레드가 동시에 동작하는 자동화 프로그램

전체 프로그램 요구사항

- **이번 실습에서 구현할 기능 5가지**
  - **설정 관리:** `config.json` 파일을 읽어 센서의 이상값 임계치(Threshold)를 설정
  - **로그 파싱:** 정규표현식(Regex)으로 비정형 텍스트 로그에서 `[타임스탬프]`와 `센서 값` 추출
  - **객체 지향 프로그래밍:** Day 2에서 만든 `Sensor` 클래스로 데이터를 구조화하여 관리
  - **데이터 분석:** NumPy로 평균·최댓값을 계산하고, 불리언 인덱싱으로 이상값 탐지
  - **시각화 & 자동화:** 이상값을 붉은 점으로 표시한 그래프를 저장하고, 멀티스레딩으로 분석 자동화

단계별 구현 순서

- **4단계로 나누어 점진적으로 완성**

단계	내용	핵심 기술
준비	설정 파일(`json`) 및 샘플 로그 파일(`txt`) 생성	`json`, 파일 I/O
Step 1	JSON 설정 파일 로드 및 정규표현식 패턴 설계	`json`, `re`
Step 2	`Sensor` 클래스 통합 및 로그 데이터 파싱 구현	OOP, `re`
Step 3	NumPy 이상값 탐지 및 Matplotlib 시각화	`numpy`, `matplotlib`
Step 4	Threading 자동화 및 최종 통합	`threading`

Part 2. Step 1 — JSON 설정 로드 & 정규 표현식

Step 1 개념 — JSON 설정 관리

- **하드코딩 대신 외부 파일에서 설정을 읽어오는 구조**

  - **왜 JSON으로 설정을 관리하는가?**
    - 코드를 수정하지 않고 임계치 값만 바꿀 수 있음
    - 로봇 종류나 환경마다 다른 설정 파일을 사용할 수 있음
    - 팀원과 설정값을 공유·버전 관리할 수 있음

  - **config.json 파일 구조 예시**

```json
{
    "sensor_name": "Lidar_1",
    "threshold": 80.5
}
```

Step 1 개념 — 정규표현식(Regex) 패턴 설계

- **비정형 텍스트 로그에서 데이터를 추출하는 '규칙'**

  - **로그 파일 한 줄 예시**

```
[2024-05-20 12:00:01] INFO: Sensor value = 45.2
```

  - **정규표현식 패턴 분석**

패턴 조각	의미	추출 대상
`\[(?P<timestamp>.*?)\]`	`[...]` 안의 문자열 캡처	타임스탬프
`.*?`	임의의 문자열 (최소 매칭)	`INFO:` 부분 스킵
`value =`	리터럴 문자열	구분자
`(?P<value>\d+\.\d+)`	소수점 숫자 캡처	센서 값

Step 1 — 완성 코드

- **설정 파일을 읽고 정규표현식 패턴을 정의하는 단계**

```python
import json
import re

# 1. 설정 파일 읽기 (임계치 설정)
def load_config():
    # JSON 파일 생성 및 로드
    config_data = {"sensor_name": "Lidar_1", "threshold": 80.5}
    with open('config.json', 'w') as f:
        json.dump(config_data, f)

    with open('config.json', 'r') as f:
        return json.load(f)

# 2. 정규표현식 정의
# 로그 형식: "[2024-05-20 12:00:01] INFO: Sensor value = 45.2"
log_pattern = r"\[(?P<timestamp>.*?)\] .*? value = (?P<value>\d+\.\d+)"

config = load_config()
print(f"설정 로드 완료: 임계치 {config['threshold']}")
```

Part 3. Step 2 — Sensor 클래스 통합 & 로그 파싱

Step 2 개념 — OOP로 센서 데이터 구조화

- **Day 2에서 배운 클래스를 활용하여 로봇 하드웨어를 소프트웨어로 추상화**
  - **Sensor 클래스의 역할**
    - 센서 이름, 측정값 목록, 타임스탬프 목록을 하나의 객체로 관리
    - `add_reading()` 메서드로 파싱된 데이터를 순서대로 추가
  - **파싱 흐름**

```
로그 파일 한 줄 읽기
      ↓
re.search()로 패턴 매칭
      ↓
match.group()으로 데이터 추출
      ↓
sensor_obj.add_reading()으로 저장
```

Step 2 — 완성 코드

- **Sensor 클래스와 파싱 함수를 작성하는 단계**

```
import numpy as np

# Day2에서 만든 Sensor 클래스 (간략화 버전)
class Sensor:
    def __init__(self, name):
        self.name = name
        self.data = []
        self.timestamps = []

    def add_reading(self, timestamp, value):
        self.timestamps.append(timestamp)
        self.data.append(float(value))

def parse_logs(file_path, sensor_obj):
    with open(file_path, 'r') as file:
        for line in file:
            match = re.search(log_pattern, line)
            if match:
                # 정규표현식 그룹에서 데이터 추출
                ts = match.group('timestamp')
                val = match.group('value')
                sensor_obj.add_reading(ts, val)
```

Step 2 핵심 포인트 — `if match:` 가 중요한 이유

- `re.search()` 가 패턴을 찾지 못하면 `None` 을 반환
  - 잘못된 코드 (AttributeError 발생)

```
match = re.search(log_pattern, line)
ts = match.group('timestamp')  # ← match가 None이면 오류!
```

  - 올바른 코드 (항상 None 체크)

```
match = re.search(log_pattern, line)
if match:                          # ← None 여부 먼저 확인
    ts = match.group('timestamp')
    val = match.group('value')
```

  - 규칙: `re.search()` / `re.match()` 결과는 항상 `if match:` 로 확인 후 사용

Part 4. Step 3 — NumPy 이상값 탐지 & Matplotlib 시각화

Step 3 개념 — NumPy 불리언 인덱싱으로 이상값 탐지

- **리스트 대신 NumPy 배열을 사용하면 벡터화 연산으로 이상값을 한 번에 탐지**
  - **Python 리스트 방식 (비효율)**

```
anomalies = [v for v in data if v > threshold]  # 루프 필요
```

  - **NumPy 방식 (벡터화 연산)**

```
values = np.array(sensor_obj.data)    # 배열로 변환
anomalies = values > threshold         # 불리언 배열 생성
anomaly_indices = np.where(anomalies) # 이상값 인덱스 추출
```

  - `np.where(condition)` → 조건이 `True`인 인덱스를 튜플로 반환

Step 3 — 완성 코드 (분석 & 시각화)

```python
import matplotlib.pyplot as plt

def analyze_and_plot(sensor_obj, threshold):
    # 리스트를 NumPy 배열로 변환 (고속 연산 가능)
    values = np.array(sensor_obj.data)

    # 이상값 탐지: NumPy 불리언 인덱싱 활용
    anomalies = values > threshold
    anomaly_indices = np.where(anomalies)

    # 그래프 그리기
    plt.figure(figsize=(10, 5))
    plt.plot(sensor_obj.data, label='Sensor Value', color='blue')

    # 이상값 표시 (붉은 점)
    if len(anomaly_indices[0]) > 0:
        plt.scatter(anomaly_indices, values[anomaly_indices],
                    color='red', label='Anomaly')

    plt.axhline(y=threshold, color='green', linestyle='--', label='Threshold')
    plt.title(f"Robot {sensor_obj.name} Data Analysis")
    plt.xlabel("Time Steps")
    plt.ylabel("Value")
    plt.legend()

    # 결과 저장
    plt.savefig('sensor_report.png')
    plt.show()
    print(f"분석 완료: 이상값 {len(anomaly_indices[0])}개 발견. 'sensor_report.png' 저장됨.")
```

Step 3 시각화 결과 해석

- `sensor_report.png` **에서 확인할 수 있는 정보**
  - **파란 선 (Sensor Value):** 시간 흐름에 따른 전체 센서 값 변화
  - **초록 점선 (Threshold):** 설정 파일에서 읽어온 임계치 선
    - `plt.axhline()` — 가로 기준선 그리기
  - **붉은 점 (Anomaly):** 임계치를 초과한 이상값 위치
    - `plt.scatter()` — 산점도로 특정 지점 강조 표시
  - `plt.savefig()` vs `plt.show()`
    - `savefig()`: 파일로 저장 (스레드 안에서도 안전)
    - `show()`: 화면에 창을 표시 (반드시 메인 스레드에서 실행)

Part 5. Step 4 — 멀티스레딩 자동화

Step 4 개념 — 왜 멀티스레딩이 필요한가?

- **로봇은 로그 분석과 주행 제어를 동시에 수행해야 함**

  - **단일 스레드 방식의 문제**

```
[메인] 로그 분석 시작 → (분석 완료까지 대기) → [메인] 주행 제어
```

- 분석이 끝날 때까지 주행 제어가 멈춤 → 로봇이 정지!

  - **멀티스레딩 방식**

```
[메인 스레드]   주행 제어 계속 실행 ──────────────────────────
[백그라운드]                          로그 분석 → 시각화 → 저장
```

- 두 작업이 동시에 진행됨

Step 4 — 완성 코드 (멀티스레딩 & 통합)

```python
import threading
import time

def automation_task(sensor_name, threshold):
    print("배경 분석 스레드 시작...")
    robot_sensor = Sensor(sensor_name)
    # 실제 환경에서는 실시간으로 생성되는 로그를 감시
    parse_logs('robot_log.txt', robot_sensor)
    analyze_and_plot(robot_sensor, threshold)

if __name__ == "__main__":
    # 샘플 로그 데이터 생성 (실습용)
    with open('robot_log.txt', 'w') as f:
        for i in range(20):
            val = 50 + (np.random.rand() * 50)  # 50~100 사이 값
            f.write(f"[2024-05-20 12:00:{i:02d}] INFO: Sensor value = {val:.1f}\n")

    # 스레드 생성 및 시작
    thread = threading.Thread(
        target=automation_task,
        args=(config['sensor_name'], config['threshold'])
    )
    thread.start()

    print("메인 스레드: 로봇의 다른 작업(예: 주행 제어)을 계속 수행합니다...")
    thread.join()  # 스레드 종료 대기
    print("프로그램 종료.")
```

Step 4 핵심 포인트 — Thread API

- `threading.Thread` 의 주요 매개변수와 메서드

  - **스레드 생성**

```
thread = threading.Thread(target=함수명, args=(인자1, 인자2))
```

매개변수	역할
`target`	스레드에서 실행할 함수
`args`	함수에 전달할 인자 (튜플)

  - **주요 메서드**

메서드	역할
`thread.start()`	스레드 시작 (백그라운드에서 `target` 함수 실행)
`thread.join()`	스레드가 끝날 때까지 메인 스레드 대기

Part 6. Troubleshooting & 심화 미션

자주 발생하는 오류 — JSONDecodeError & AttributeError

- **오류 1: JSONDecodeError**
  - **원인:** `config.json` 파일 형식이 잘못되었거나(콤마 누락 등) 파일이 비어 있음
  - **해결:**

```
try:
    with open('config.json', 'r') as f:
        config = json.load(f)
except json.JSONDecodeError:
    print("JSON 형식 오류 — 기본값 사용")
    config = {"sensor_name": "Lidar_1", "threshold": 80.5}
```

- **오류 2: AttributeError in Regex**
  - **원인:** `re.search()`가 `None`을 반환했는데 `.group()` 호출
  - **해결:** `if match:` 구문으로 항상 None 체크 후 사용

자주 발생하는 오류 — Matplotlib Thread Error & ModuleNotFoundError

- **오류 3: Matplotlib GUI/Thread Error**
  - **원인:** 일부 OS에서 `plt.show()`를 메인 스레드가 아닌 곳에서 호출하면 에러 발생
  - **해결:** 스레드 내부에서는 `savefig()`만 사용, `show()`는 메인 스레드에서 호출

```
# 스레드 내부 (안전)
plt.savefig('sensor_report.png')

# 메인 스레드에서 (화면 표시)
plt.show()
```

- **오류 4: ModuleNotFoundError**
  - **원인:** `numpy` 또는 `matplotlib`이 설치되지 않음
  - **해결:**

```
pip install numpy matplotlib
```

심화 미션

- **기본 실습을 완료한 후 도전해 보세요**

  - **미션 1: 실시간 로그 감시 (Live Tail)**
    - 로그 파일에 새로운 줄이 추가될 때마다 실시간으로 파싱하여 이상값 탐지
    - 이상값 발견 시 터미널에 경고 메시지 출력
    - 힌트: `time.sleep()` + 파일 끝까지 읽고 새 줄을 기다리는 루프

  - **미션 2: 필터링 기능 추가**
    - 정규표현식으로 로그 등급이 `ERROR`인 줄만 추출
    - 추출한 에러 로그를 `error_logs.json` 파일로 저장

  - **미션 3: 다중 센서 관리**
    - 딕셔너리로 여러 개의 `Sensor` 객체를 관리하고 각각 다른 임계치 적용
    - 모든 센서의 결과를 하나의 통합 리포트로 작성

Part 7. Day 1~3 마무리 & 다음 단계

3일간의 핵심 개념 정리 — Day 1 & Day 2

- **Day 1: 기초 다지기와 환경 구축**
  - **가상환경과 VS Code:** `venv`를 활용한 환경 격리의 중요성과 IDE 도구 설정
  - **제어 흐름(Control Flow):** 로봇의 '판단' 기준이 되는 `if`, `for`, `while`의 작동 원리
  - **함수 정의:** 복잡한 로직을 재사용 가능한 단위로 캡슐화하는 `def` 키워드
- **Day 2: 객체지향과 데이터 다루기**
  - **객체지향 프로그래밍(OOP):** 로봇의 모터·센서를 모델링하기 위한 **클래스와 상속, 캡슐화**
  - **NumPy 수치 계산:** 리스트보다 강력한 **벡터화 연산**으로 센서 데이터를 효율적으로 처리
  - **데이터 시각화:** `Matplotlib`으로 수치 데이터를 그래프로 그려 로봇 상태를 직관적으로 파악

3일간의 핵심 개념 정리 — Day 3

- **Day 3: 데이터 관리와 병렬 처리**
  - **JSON과 설정 관리**: 하드코딩 대신 외부 파일에서 설정을 불러오는 **데이터 직렬화**
  - **정규 표현식(Regex)**: 방대한 로그 파일에서 필요한 패턴의 데이터를 **추출 및 분석**
  - **멀티스레딩(Threading)**: 분석 스레드와 메인 루프를 분리하여 **병렬 처리**

3일 과정 최종 체크리스트 (1/4) — 환경 및 제어 흐름

- **환경 및 기초**
  - [ ] VS Code에서 가상환경(venv)을 생성하고 인터프리터를 선택할 수 있는가?
  - [ ] `pip install`을 통해 필요한 외부 패키지를 설치할 수 있는가?
  - [ ] `if-elif-else`를 사용하여 여러 조건에 따른 로직을 짤 수 있는가?
  - [ ] `for` 문과 `range()`를 조합해 원하는 횟수만큼 반복할 수 있는가?
  - [ ] `while` 루프를 사용해 지속적으로 동작하는 제어문을 만들 수 있는가?
- **함수 및 데이터 구조**
  - [ ] 매개변수와 반환값이 있는 함수를 정의하고 호출할 수 있는가?
  - [ ] 리스트(List)와 딕셔너리(Dict)의 차이를 이해하고 데이터를 저장할 수 있는가?
  - [ ] 함수의 가독성을 위해 Docstring을 작성할 수 있는가?

3일 과정 최종 체크리스트 (2/4) — 객체지향(OOP)

- **객체지향(OOP)**
  - [ ] `class` 키워드로 클래스를 정의하고 인스턴스를 생성할 수 있는가?
  - [ ] `__init__` 메서드에서 초기 변수를 설정할 수 있는가?
  - [ ] 상속을 통해 부모 클래스의 기능을 자식 클래스가 이어받을 수 있는가?
  - [ ] 메서드 오버라이딩을 통해 기능을 재정의할 수 있는가?
- **수치 계산 및 분석**
  - [ ] NumPy 배열을 생성하고 행렬 연산을 수행할 수 있는가?
  - [ ] Matplotlib을 사용하여 데이터를 선 그래프나 산점도로 그릴 수 있는가?
  - [ ] JSON 파일을 읽고(load) 쓰는(dump) 법을 알고 있는가?

3일 과정 최종 체크리스트 (3/4) — 고급 도구 및 병렬 처리

- **고급 도구 및 병렬 처리**

  - [ ] 정규 표현식의 기본 문법(`\d`, `\w`, `*` 등)을 이해하고 있는가?
  - [ ] `re` 모듈을 사용하여 텍스트에서 특정 패턴을 찾아낼 수 있는가?
  - [ ] 멀티스레딩을 사용하여 작업을 병렬로 실행할 수 있는가?
  - [ ] 스레드 간 안전한 데이터 공유를 위한 방법을 알고 있는가?
  - [ ] `if __name__ == "__main__":` 구문의 목적을 이해하고 적용했는가?

3일 과정 최종 체크리스트 (4/4) — 오늘 실습 핵심

- **오늘 실습 핵심 확인**

  - [ ] `config.json`에서 임계치를 읽어 프로그램에 적용할 수 있는가?
  - [ ] 정규표현식으로 로그 파일에서 타임스탬프와 센서 값을 추출할 수 있는가?
  - [ ] `Sensor` 클래스에 파싱된 데이터를 추가하고 관리할 수 있는가?
  - [ ] NumPy 불리언 인덱싱으로 임계치를 초과한 이상값을 탐지할 수 있는가?
  - [ ] Matplotlib으로 이상값이 표시된 그래프를 파일로 저장할 수 있는가?
  - [ ] `threading.Thread`로 분석 작업을 백그라운드에서 실행할 수 있는가?
  - [ ] `thread.start()`와 `thread.join()`의 차이를 설명할 수 있는가?